

**Comparative Analysis of Sequential Models for One-Step-Ahead Mobile Network Traffic
Forecasting**

G. Keane

African Leadership University

Machine Learning Techniques I

20 September 2026

Abstract

Capacity assignment, overnight cell sleep scheduling, and anomaly alarms all act on a ten-minute horizon, and each acts on a forecast of the next slot. This study compares three architecturally distinct sequential models for one-step-ahead forecasting of mobile Internet traffic on the Telecom Italia Milan grid, and asks whether their ranking survives a change of geographical area. A chunked streaming aggregation reduced peak memory growth per daily file from 760.8 MiB to 177.3 MiB, a 4.3-fold reduction with byte-identical output. SARIMA, a long short-term memory network, and a one-dimensional convolutional network were fitted per area on an identical 144-slot window and evaluated over 16 to 22 December 2013 against persistence and seasonal-naïve references. The ranking was stable, LSTM first, then CNN, then SARIMA by mean absolute error on all five areas tested, but the references reframe it: SARIMA was beaten by persistence on every area by 12% to 24% of mean absolute error, the CNN lost on two of five, and on the highest-traffic area no model beat persistence on mean absolute percentage error. Only the LSTM improved on persistence everywhere, by 4.2% to 10.4%, at 19 times the CNN's training cost for gains between 1.8% and 18.8%. A multi-seasonal decomposition placed 82.9% of the variance in the daily cycle and 10.0% in the weekly cycle, which the 144-slot window cannot reach; weekends were accordingly the worst day of the evaluation week for 13 of the 15 area-model pairs. The convolutional model additionally over-predicted the overnight trough on the area whose trough was deepest, reaching 36.8% mean percentage error between 03:00 and 05:00 against 6.8% in daylight.

Keywords: time series forecasting, mobile network traffic, LSTM, convolutional neural network, SARIMA, baseline comparison

Comparative Analysis of Sequential Models for One-Step-Ahead Mobile Network Traffic Forecasting

A mobile operator reads its network in ten-minute slots. Capacity assignment to a cell, the decision to put a base station to sleep overnight, and the alarm that flags an outage all act on that horizon, and each acts on a forecast of the next slot. The margin between a good and a bad forecaster there is small in absolute terms and expensive in aggregate, which is a reason to measure it carefully before assuming that a larger model wins.

This study uses the Telecom Italia Milan activity release [6]: the city partitioned into 10,000 squares, with Internet activity recorded every ten minutes from 1 November 2013 to 1 January 2014. The research question is fixed by the assignment brief and quoted verbatim:

How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?

Formally, for area a and interval t , the forecaster receives the history of traffic up to and including t and emits an estimate of $x_a(t+1)$. Three architecturally distinct models (SARIMA, an LSTM, and a 1D-CNN) were compared on the three highest-traffic squares over the held-out week of 16 to 22 December 2013, with two further squares carried through the same pipeline as a lower-traffic contrast.

Two decisions set the terms of the comparison. The first is the persistence reference, $\hat{x}(t+1) = x(t)$. Without it, three error figures in a table say nothing about whether any model is worth running. The second is that the exploratory analysis earns its place by supplying the modelling constraints: the weekly period identified in the exploratory section is the same period that explains which days the models lose on in the results.

Related Work

Three Lines of Evidence, Three Model Families

Work on cellular and wireless traffic forecasting divides along three lines, and each supplied one of the models compared here.

Seasonal linear models remain competitive where periodicity is stable. Kochetkova et al. [1] fitted SARIMA and Holt-Winters to hourly volumes from a Portuguese operator and reported SARIMA at roughly 11% mean absolute percentage error (MAPE) on download traffic against 15% for Holt-Winters. Their setting is univariate, strongly diurnal, and short-horizon, which matches the profile found in the exploratory analysis below, and it is why SARIMA belongs here as a serious classical baseline.

Recurrent models help where the dynamics are nonlinear. Azari et al. [2] compared LSTM and ARIMA on real network traces and found LSTM ahead when histories are long and patterns nonlinear, with ARIMA still competitive under light traffic and tight compute budgets. They treat the advantage as conditional instead of universal, which is the framing adopted here, and the cost analysis below puts a number on the condition in training seconds.

No architecture dominates every site. Sone et al. [3] compared Holt-Winters, SARIMA, LSTM, GRU, CNN, and hybrid models across 470 enterprise access points and found that the ranking moves with the series. This study therefore evaluates five areas and reports a ranking for each one.

How This Study Differs From Prior Work on the Same Data

Two studies use the same Milan corpus. Zhang and Patras [4] fused ConvLSTM and 3D-ConvNet modules for network-wide, multi-step forecasting, exploiting spatial structure across the whole grid. Santos et al. [5] clustered cells by intensity and compared LSTM against GRU on cluster-aggregated series, finding LSTM ahead in most clusters with performance varying geographically. The present study is deliberately narrower than both: univariate, single-area, one-step-ahead, with three models chosen for differing inductive bias. The shared dataset provides continuity; the task, the model set, and the comparative question differ. Zhang and Patras [4] also

supply the evidence that convolution extracts useful structure from this signal, which justifies the 1D-CNN here as a temporal-only analogue of that idea.

The Three Selected Models

The exploratory analysis constrains the choice more sharply than the literature does. It finds a daily period at lag 144 carrying 82.9% of the variance on the busiest area, a weekly period at lag 1008 worth a further 10.0%, stationarity under an augmented Dickey-Fuller test, and a spatial distribution in which the top area carries 46 times the median.

Table 1

Selected Models, the Exploratory Finding Each Responds To, and the Limitation Each Carries

Model	Inductive bias	Exploratory hook	Known limitation
SARIMA(p,d,q) $\times$ (0,1,0,144)	Linear, explicitly seasonal, parametric	Daily period dominates the ACF; series is stationary	One seasonal period only; lag 1008 unmodelled. Unconstrained on a non-negative quantity
LSTM	Gated recurrence, nonlinear learned memory	Nonlinear amplitude changes; short-lag PACF structure	Opaque and costly; a 144-step window cannot reach lag 1008
1D-CNN	Local convolution over recent lags, non-recurrent	Strong short-range dependence in the PACF	Receptive field bounded by kernel width and depth; no explicit weekly pathway

Note. ACF = autocorrelation function; PACF = partial autocorrelation function. The three models differ in kind, not in degree. They do share one blind spot: none has a path to the weekly period, and the failure analysis shows that the shared blind spot produces a shared failure mode.

Dataset and Data Preparation

The Data

Sixty-two daily tab-separated files cover 1 November 2013 to 1 January 2014. Each row carries a square identifier, a millisecond timestamp, a country code, and activity fields for SMS in and out, calls in and out, and Internet. Internet activity for a given square and timestamp is split across country codes, so constructing the tidy series requires summing those rows. That aggregation is also where the memory saving comes from.

Memory-Efficient Loading

The naive approach reads a whole day file into a data frame and then groups it. The loader used here instead reads only the three needed columns, aggregates across country codes within each 500,000-row chunk, re-aggregates across chunk boundaries, and downcasts to 16-bit unsigned integers and 32-bit floats before writing compressed Parquet.

Both paths were measured on the same day file in separate spawned subprocesses, so that allocator retention from one run could not contaminate the other, with peak resident memory sampled by a background thread.

Table 2

Peak Resident Memory Growth and Wall Time, Naive Against Chunked Aggregation

Measurement	Naive	Chunked	Change
RSS growth during load	760.8 MiB	177.3 MiB	−583.4 MiB (−76.7%)
Peak RSS	828.5 MiB	245.1 MiB	−583.4 MiB
Retained frame memory	32.96 MiB	19.23 MiB	−13.73 MiB (−41.7%)
Wall time	4.80 s	4.14 s	1.16× faster
Output rows	1,439,946	1,439,946	identical

Note. RSS = resident set size. Measured on the file of 1 November 2013. Both variants ran in freshly spawned subprocesses so that neither inherited the other's retained heap. Reproduced by `make bench`, which writes `report/memory_benchmark.json`.

The 4.3-fold reduction in memory growth is what makes the full 62-file pass comfortable on a 16 GiB machine. Peak memory under the chunked path is set by the chunk size, not by the number of files: each day is streamed into one open Parquet writer, and nothing larger than a chunk plus the running per-square aggregate is ever resident. Under the naive path each day transiently claims 0.74 GiB, which leaves little headroom once the operating system, the Python runtime, and a notebook kernel are also resident. The resulting tidy file holds 89,127,473 rows in 358 MiB.

This comes with trade-offs. The 16-bit cast for the square identifier is safe only because the grid is fixed at 10,000 squares; a larger grid would overflow silently, so this is a cast that

depends on the grid staying fixed. The 32-bit float cast costs about seven significant digits, which is immaterial against activity values in the hundreds but would not be against a series differenced many times. Chunked aggregation also requires a cross-chunk re-aggregation step, which is one more place to make an error than a single grouping operation; the benchmark asserts identical output row counts between the two paths precisely to catch that. One limitation of the evidence should be acknowledged: this is a single-day measurement. Peak memory for the full pass was not instrumented, and because the streaming writer ties peak memory to chunk size, the per-day figure is the relevant one. That is an inference, not a measurement.

Splits and Normalisation

For each evaluated area, the tidy file was filtered to that area, reindexed onto a regular ten-minute Europe/Rome grid, and split chronologically.

Table 3

Chronological Splits of the Observation Period

Split	Period	Slots	Role
Train	1 Nov to 8 Dec 2013	5,472	Parameter estimation; scaler statistics
Validation	9 to 15 Dec 2013	1,008	Hyperparameter selection, early stopping
Test	16 to 22 Dec 2013	1,008	Held out; the assignment's evaluation week

Note. No target in one split was used to fit anything evaluated on a later split.

Normalisation used a standard score computed from training-only mean and standard deviation, applied unchanged to validation and test. Targets stayed strictly inside their own split; the history feeding a window was allowed to reach back across a split boundary, which is correct for one-step-ahead forecasting, because at test time an operator knows everything up to t .

Exploratory Analysis

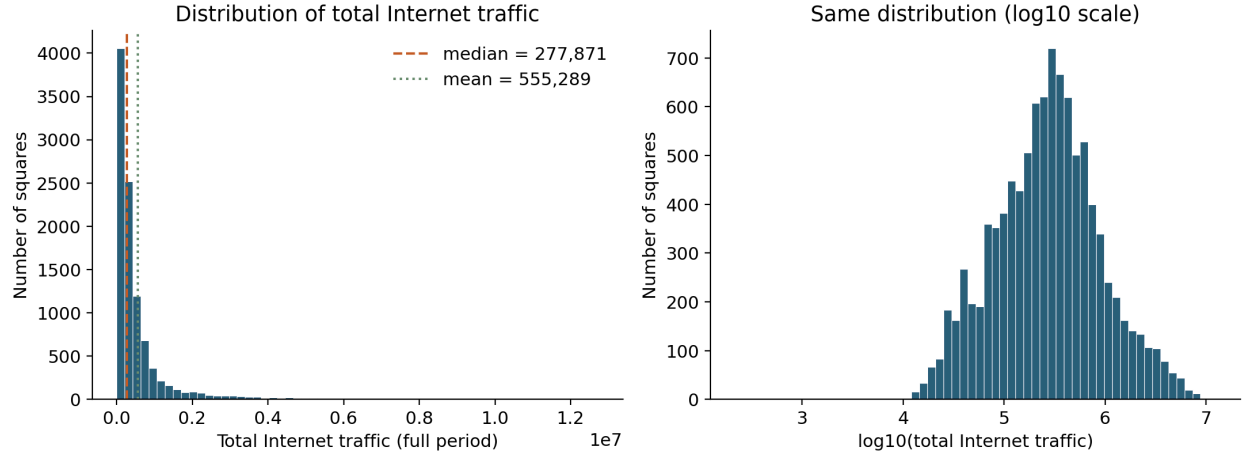
The Distribution of Traffic Across Areas

Total Internet traffic over the two months is strongly right-skewed, with a mean of 555,289 against a median of 277,871, a skewness of 4.27, and a top-to-median ratio of 45.8. The

three highest-traffic areas are squares 5161, 5059, and 5259.

Figure 1

Total Internet Traffic per Square Over the Observation Period

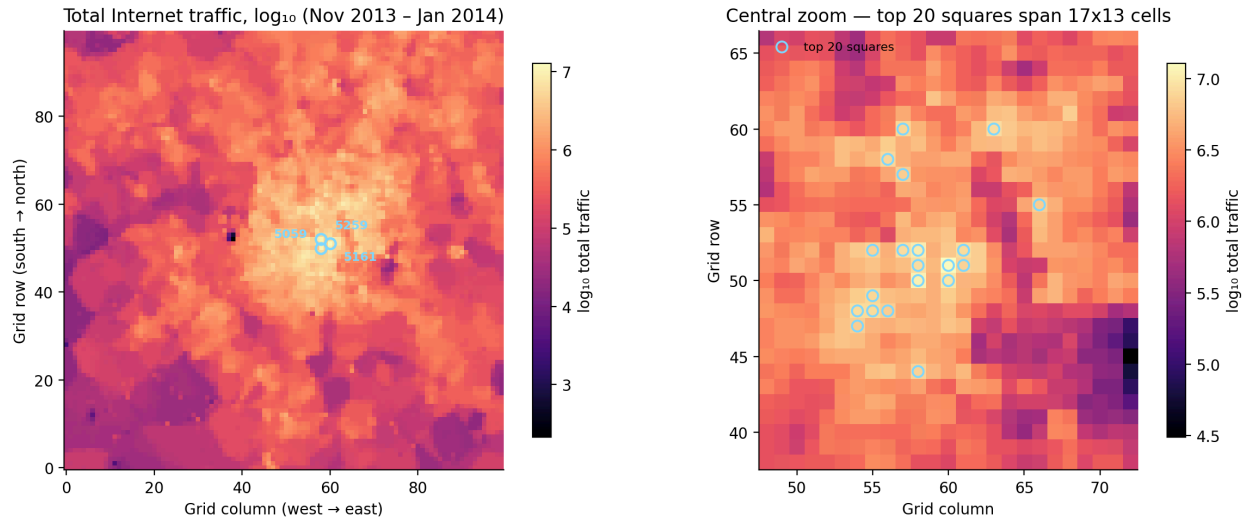


Note. Left panel on a linear axis, right panel on a base-10 logarithmic axis. The mean sits well above the median and the upper tail is long, so absolute error will concentrate on a handful of areas.

Because absolute error scales with level, mean absolute error (MAE) and root mean squared error (RMSE) are not comparable across areas; MAPE is the cross-area metric, and it is used that way in the results.

Spatial Concentration

Mapping the 10,000 totals onto the 100 by 100 lattice, with square i at row $\lfloor (i-1)/100 \rfloor$ from the south edge and column $(i-1) \bmod 100$ from the west, shows that the skew is not scattered noise but a single contiguous core.

Figure 2*Total Internet Activity Across the Milan Grid*

Note. Base-10 logarithmic scale, with the three evaluation areas marked, and a zoom on the central core. The three required areas are neighbours inside one core, so their differing weekly regimes are the finding here.

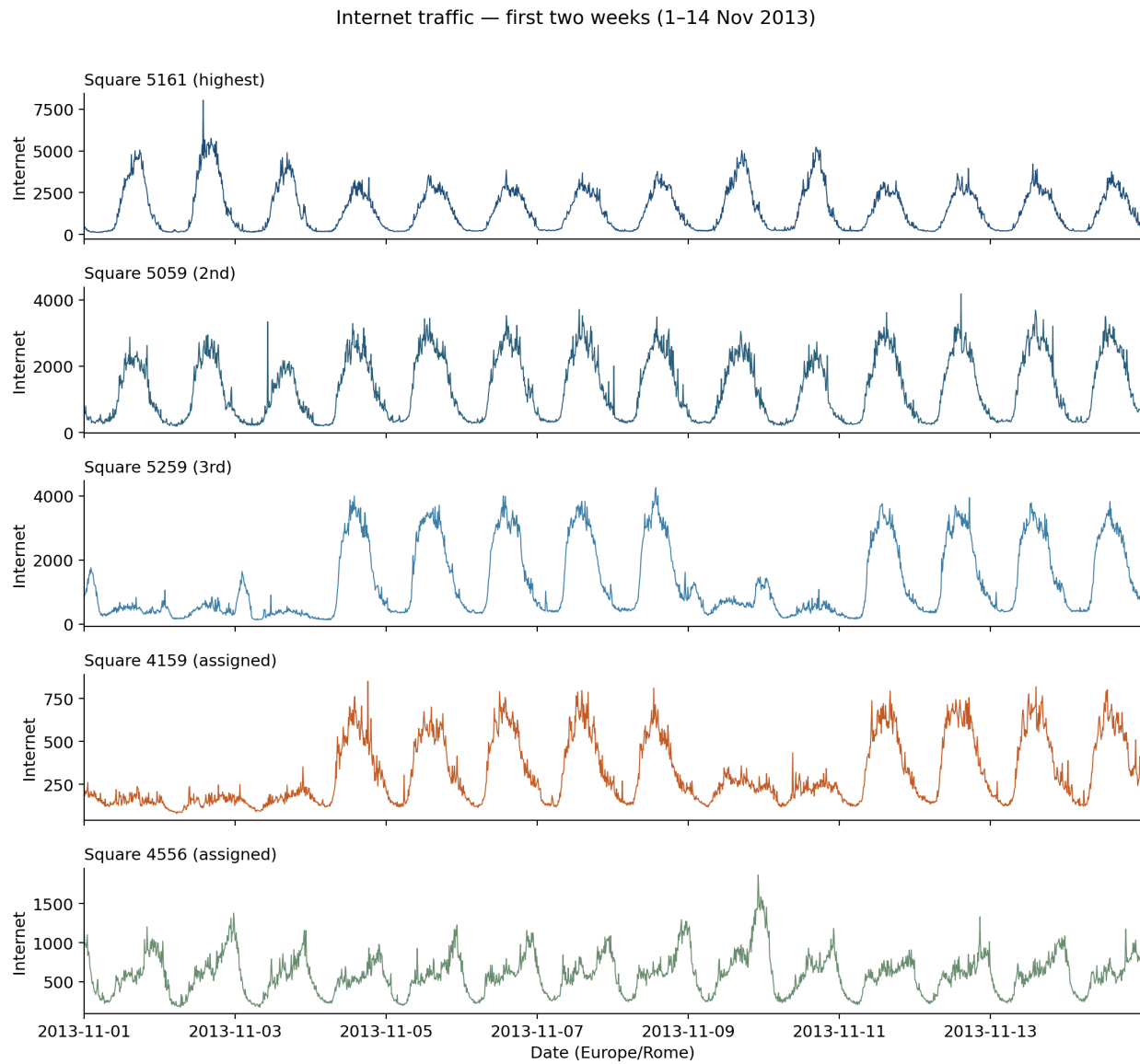
Global Moran's I on logged totals is 0.943 under rook contiguity [9], against an expectation of -0.0001 under spatial randomness: adjacent squares are almost perfectly alike. The top 20 squares fit inside a block of 17 rows by 13 columns, 2.21% of the grid, and the three evaluated areas are neighbours within it.

That has a consequence for the study's framing. The three required areas are not independent samples of high traffic. They are three cells of one city centre, so a ranking that holds across all three is evidence about one regime observed three times. Squares 4159 and 4556 were carried through the full pipeline for that reason, as a contrast.

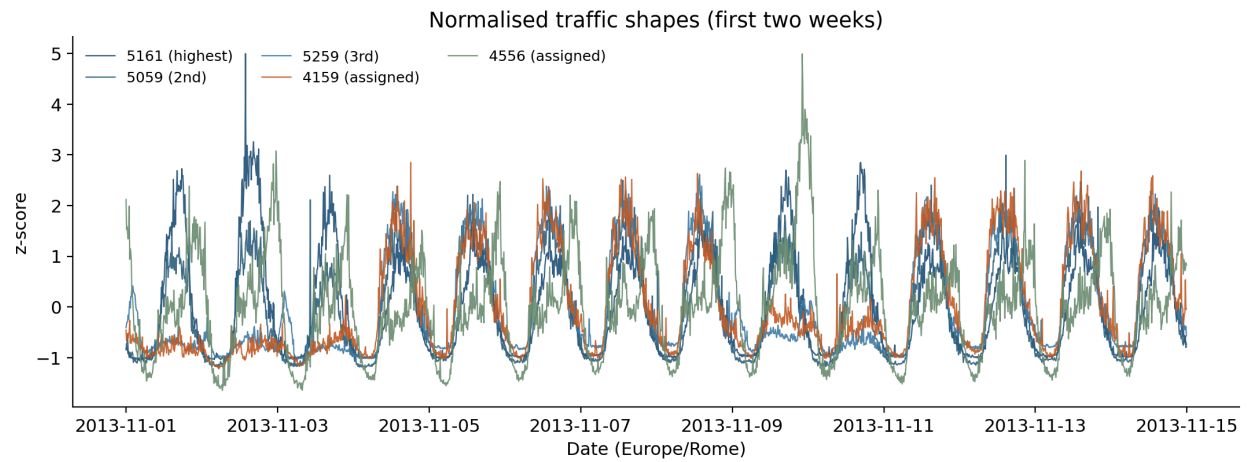
Temporal Dynamics Over the First Two Weeks

Figure 3

Internet Traffic for Five Areas, 1 to 14 November 2013



Note. The three highest-traffic areas and squares 4159 and 4556. Every series carries a clean diurnal cycle; the differences lie in amplitude and in the weekly rhythm.

Figure 4*The Same Fortnight, Standardised per Area*

Note. Each series z-scored against its own mean and standard deviation so that shape can be compared independently of level. The diurnal cycles are close to in phase, which means level alone does not separate these areas.

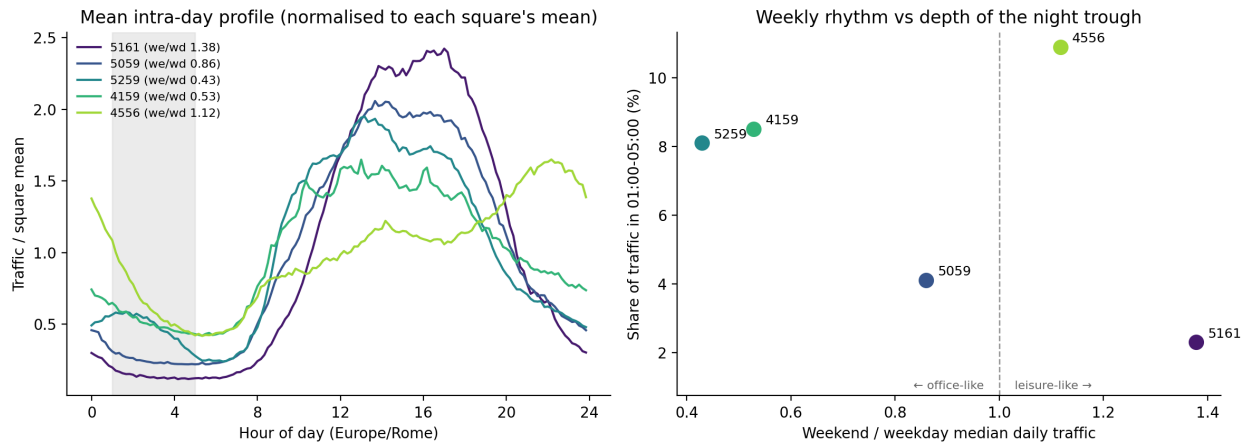
Two Regimes Among Five Neighbouring Areas

Splitting ordinary weeks (1 November to 20 December, excluding the Christmas shutdown) into weekday and weekend medians separates the five areas cleanly.

Table 4*Weekly Rhythm and Depth of the Night Trough by Area*

Area	Weekday median	Weekend median	Weekend/weekday	Traffic 01:00 to 05:00	Regime
5161	195,264	269,101	1.38	2.30%	Leisure, weekend-peaking
5059	204,400	175,600	0.86	4.11%	Mixed
5259	241,269	103,480	0.43	8.11%	Office, weekday-peaking
4159	53,793	28,431	0.53	8.52%	Office, weekday-peaking
4556	78,102	87,259	1.12	10.90%	Residential, leisure

Note. Medians of daily totals over ordinary weeks only. Squares 5161 and 5259 are neighbours on the grid with opposite weekly regimes.

Figure 5*Intra-Day Profiles and the Regime Separation*

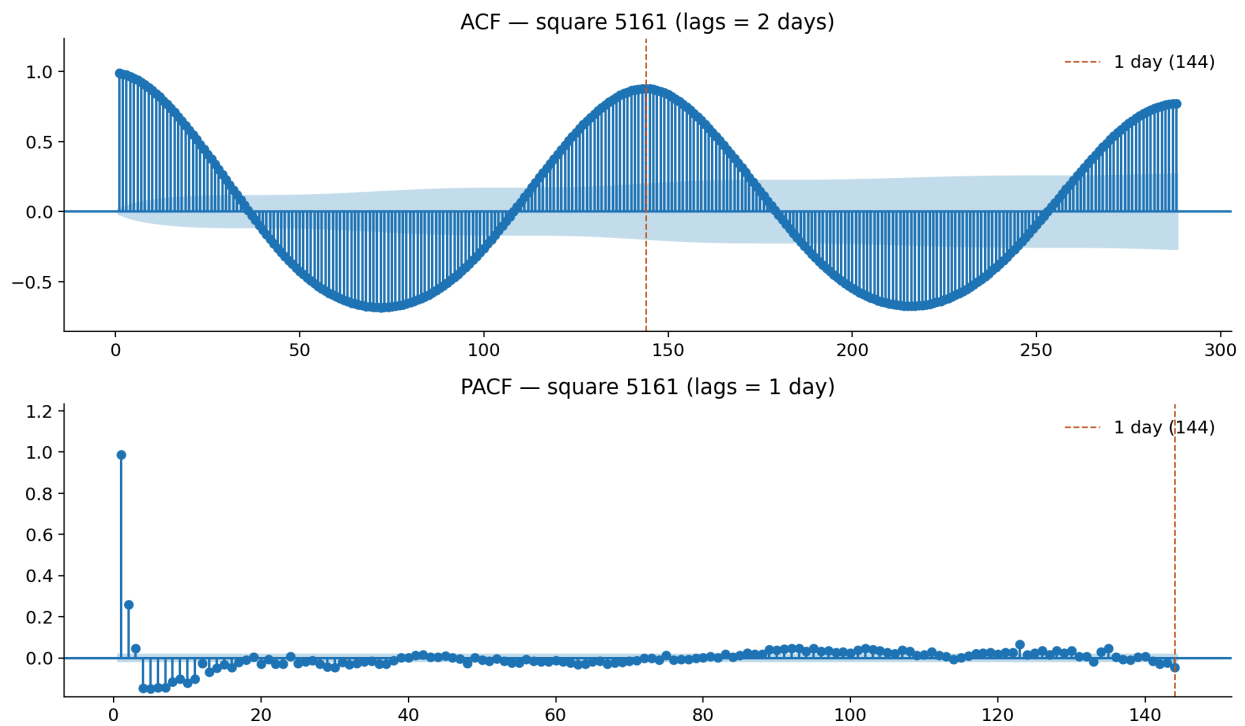
Note. Left panel: the mean intra-day profile of each area normalised by its own mean, with 01:00 to 05:00 shaded. Right panel: weekly rhythm against depth of the night trough. Square 5161 has the deepest trough of the five and the strongest weekend peak; square 5259, its neighbour, is the inverse.

Square 5161 earns 38% more on a weekend day than on a weekday and drops to 13% of its own mean overnight; square 5259 loses 57% at the weekend and holds 8.1% of its traffic in the small hours. The plausible reading is a nightlife and retail district adjacent to an office district, which is what the centre of Milan is.

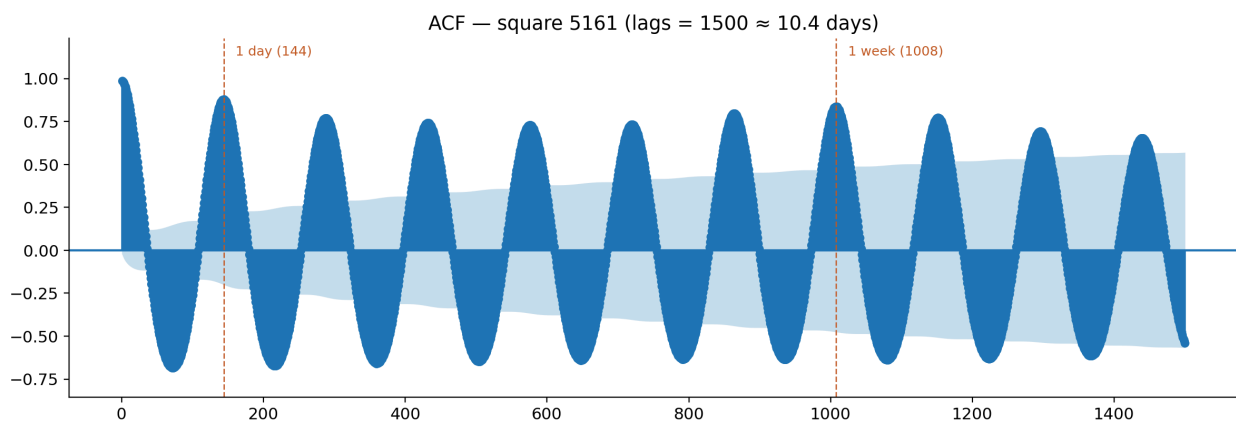
Two consequences follow. A model tuned on 5161 is tuned on the least typical of the three areas, which the transfer results test. And 5161 has the deepest night trough of the five, so because percentage error is a ratio, the same absolute mistake at 04:00 costs 5161 far more MAPE than it costs 4556. That is the mechanism behind the failure case reported below, and it was predictable from Table 4 before any model was trained.

Stationarity, Autocorrelation, and Decomposition

An augmented Dickey-Fuller test on the full 5161 series rejects the unit-root null, with a statistic of -19.03 at $p \approx 0$: the series is stationary about its seasonal mean, so aggressive differencing is not required and a seasonal-difference-plus-ARMA formulation is adequate.

Figure 6*Autocorrelation and Partial Autocorrelation for Square 5161*

Note. Autocorrelation over two days and partial autocorrelation over one day. The autocorrelation function is dominated by lag 144; the partial function emphasises short lags once that structure is accounted for.

Figure 7*Autocorrelation Extended to 1,500 Lags*

Note. The daily peak at lag 144 reaches .878; the weekly peak at lag 1008 reaches .838 and stands above its daily-multiple neighbours at lag 864 (.798) and lag 1152 (.772), so the weekly period is a real feature rather than an artefact of the daily one.

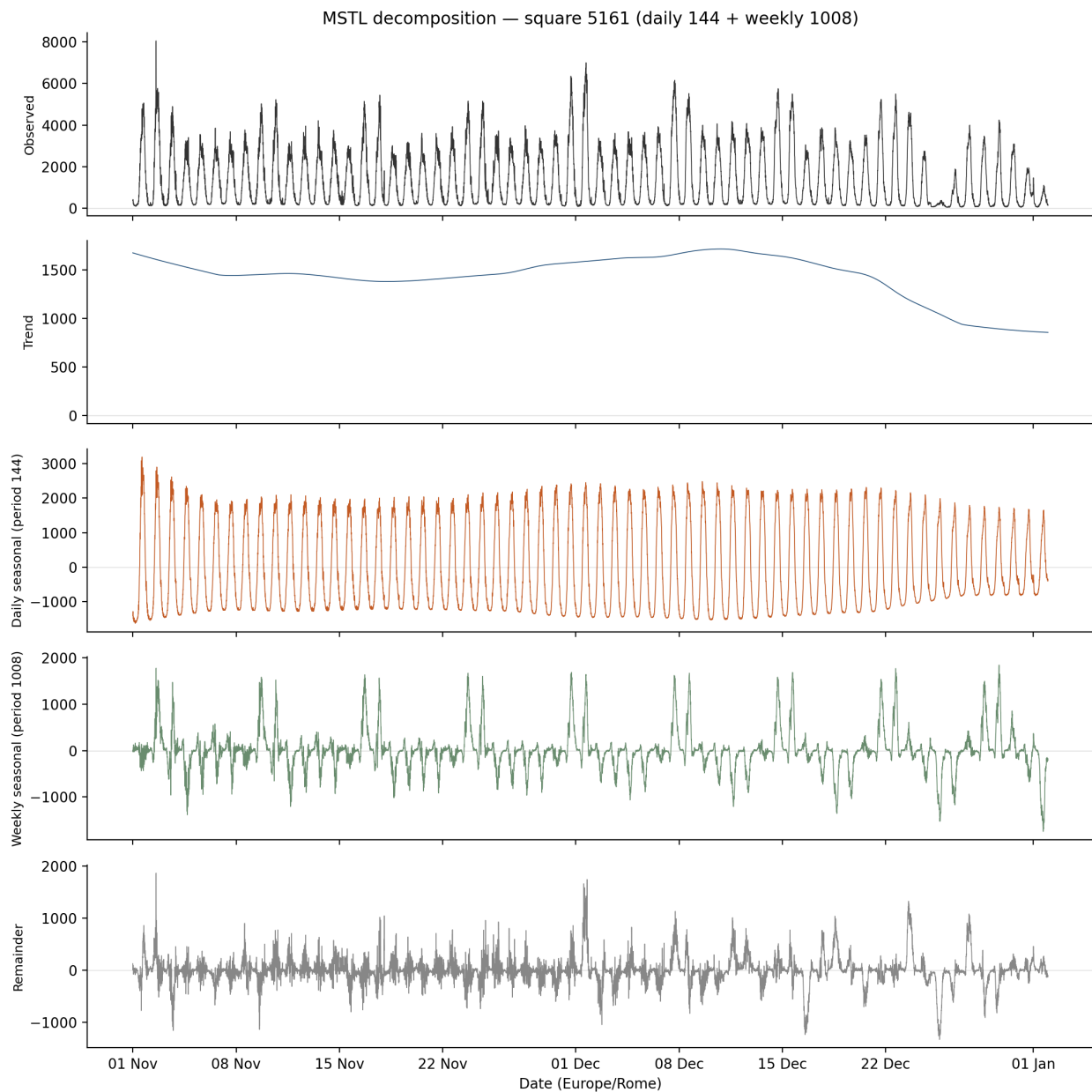
An autocorrelation peak alone does not indicate how much signal a component carries, so the series was decomposed using multiple seasonal-trend decomposition [8], fitting both periods simultaneously; a single-period decomposition [7] cannot separate them.

Table 5

Variance Shares of the Multi-Seasonal Decomposition for Square 5161

Component	Share of variance
Daily seasonal, period 144	82.94%
Weekly seasonal, period 1008	9.99%
Trend	2.76%
Remainder	3.60%

Note. Fitted at periods 144 and 1008 simultaneously.

Figure 8*Multi-Seasonal Decomposition of Square 5161*

Note. Decomposition into trend, daily seasonal component, weekly seasonal component, and remainder. The trend panel also shows the December collapse discussed under calendar effects.

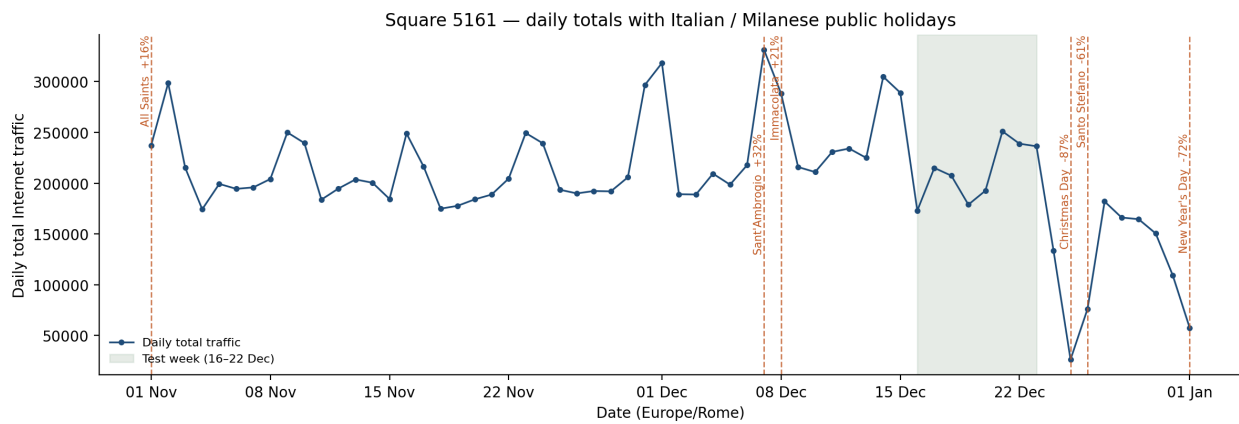
This decomposition is the most useful result in the exploratory analysis. A forecaster that captures daily seasonality perfectly and nothing else leaves about 16% of the variance on the table, and the weekly component is nearly two thirds of that. The remainder has a standard

deviation of 262 against a series standard deviation of 1,382, so the decomposition sets a scale for the residual. The observed LSTM mean absolute error of 86 is below that figure, which indicates that the LSTM absorbs some of the seasonal structure too; the remainder is not a hard error floor for a one-step forecaster that can also observe $x(t)$.

Calendar Effects

Figure 9

Daily Totals for Square 5161 With Public Holidays Marked



Note. Italian and Milanese public holidays are marked and the evaluation week is shaded. The Christmas collapse begins on 23 December, one day after the test week ends.

Table 6

Holiday Deviations on Square 5161 Against the Median Same Weekday

Date	Holiday	Weekday	Deviation
1 Nov 2013	All Saints, national	Friday	+16.1%
7 Dec 2013	Sant'Ambrogio, Milan only	Saturday	+32.2%
8 Dec 2013	Immacolata, national	Sunday	+20.6%
25 Dec 2013	Christmas Day	Wednesday	-87.1%
26 Dec 2013	Santo Stefano	Thursday	-61.1%
1 Jan 2014	New Year's Day	Wednesday	-71.8%

Note. Each holiday is compared against the median daily total for the same weekday in ordinary, non-holiday weeks.

Sant'Ambrogio is the feast of Milan's patron saint, a city-only public holiday, and the opening of the Scala season. On square 5161, the leisure area, it is the largest positive anomaly in the record, at 32% above a typical Saturday. No national holiday calendar would predict a city-only feast day, and no model here is given one.

Two features of the calendar matter for the experimental design. The 7 and 8 December pair sits inside the training window and immediately before the validation week, so the models are fitted across two consecutive anomalous days. And the Christmas collapse begins on 23 December, one day after the test week ends. The evaluation window is therefore an ordinary week by accident of the calendar, which makes the results a clean read on ordinary dynamics and says nothing about behaviour through a shutdown.

Methodology

Forecasting Setup

At each t in the test week the model receives the observed history up to t and predicts $x(t+1)$. Predictions are one-step throughout; nothing is fed back autoregressively. All metrics were computed on the original traffic scale after inverse transformation, over all 1,008 test slots. MAE is the mean absolute error, RMSE the root mean squared error, and MAPE is $100 \times \text{mean}(|\hat{y} - y| / \max(|y|, \epsilon))$ with $\epsilon = 10^{-8}$. The guard never activates: the minimum observed test value across all five areas is 94.7, so MAPE is well defined and not inflated by near-zero denominators.

Input Representation

Table 7

Input Representation and Preprocessing for Each Model

	SARIMA	LSTM	1D-CNN
Input	Raw level series	Sliding window	Sliding window
Sequence length	Full history, seasonal difference at lag 144	144 (24 h)	144 (24 h)
Tensor shape	n/a	(batch, 144, 1)	(batch, 1, 144)
Normalisation	None, operates on levels	Standard score, train-only M and SD	Standard score, train-only M and SD
Target	$x(t+1)$, level	$x(t+1)$, normalised, inverse-transformed before scoring	as LSTM
Windows, train/val/test	n/a	5,328 / 1,008 / 1,008	5,328 / 1,008 / 1,008

Note. Both neural models consume an identical window, so any gap between them is down to inductive bias.

Sequence length 144 was chosen from the autocorrelation function: it covers one full diurnal cycle and so admits same-time-yesterday dependence, the strongest single predictor after $x(t)$ itself. The decision not to extend to 1,008 was deliberate and is the study's main self-imposed constraint. A 1,008-step window would multiply LSTM training cost by roughly seven on hardware where the largest configuration already takes 151 s per fit, and would shrink the training set from 5,328 to 4,464 windows, paying twice, in compute and in data. The failure analysis shows what that choice costs.

Model Descriptions

SARIMA. Implemented as $SARIMA(p, d, q) \times (0, 1, 0, 144)$. The series is seasonally differenced at lag 144 and an $ARIMA(p, d, q)$ is fitted by state-space maximum likelihood to the differenced series. Full seasonal autoregressive and moving-average terms at a period of 144 were rejected on tractability grounds: the Kalman state grows with the seasonal period, and such a fit is impractical at ten-minute resolution on this hardware. A seasonal difference of order one at period 144 still encodes the dominant daily cycle. Test-week forecasts hold the parameters fixed, run the Kalman filter over the extended series, and invert each one-step prediction so that every step uses the true lag-144 level. Weekly structure is not represented.

LSTM. A one- or two-layer recurrent network over the 144-step window, taking the final hidden state through dropout into a linear head producing one value. Optimisation used Adam at a learning rate of .001, mean squared error on normalised targets, a batch size of 64, gradient-norm clipping at 1.0, a maximum of 40 epochs, and early stopping on validation MAE measured on the inverse-transformed scale with a patience of six epochs. Scoring early stopping on the raw scale instead of on normalised loss keeps the selection criterion identical to the reported metric.

1D-CNN. Stacked one-dimensional convolution, rectified linear activation, and dropout blocks over the 144-step window, then a linear head on the flattened feature map. The optimiser, loss, batch size, epoch cap, and early-stopping rule are identical to the LSTM, so the two neural models differ only in how they consume the window. With one layer and a kernel of five, the receptive field is five slots, or 50 minutes, and the flattening linear head is what lets the model see the rest of the window, as a fixed positional weighting, not as learned temporal structure.

Persistence and seasonal naïve. The references $\hat{x}(t+1) = x(t)$ and $\hat{x}(t+1) = x(t+1-144)$. Neither estimates parameters, so both have zero training time by construction. They are not entries in the comparison. Their job is to calibrate it.

Hyperparameter Search

Search was run on square 5161 only, selecting on validation MAE. Full grids with per-configuration scores, timings, and reasoning are in the accompanying experiment log.

Table 8

Search Grids and Selected Configurations

Model	Grid	Configurations	Selected
SARIMA	(p,d,q) over ten orders	10	$(1,0,1) \times (0,1,0,144)$
LSTM	layers $\{1,2\} \times$ hidden $\{32,64\} \times$ dropout $\{0.0,0.2\}$	8	2 layers, hidden 64, dropout 0.0, epoch 18
1D-CNN	filters $\{32,64\} \times$ kernel $\{3,5\} \times$ layers $\{1,2\}$	8	32 filters, kernel 5, 1 layer, dropout 0.1, epoch 13

Note. Every area other than 5161 reuses these configurations without a fresh search.

Three findings from the search changed what was done next, so they are reported here as well as in the log.

SARIMA plateaus immediately. Validation MAE fell from 294.6 at order (0,0,0) to 147.9 at (1,0,0) to 136.5 at (1,0,1), then moved by less than 0.2 across (2,0,1), (1,0,2), and (2,0,2) while fit time roughly doubled. The Akaike information criterion kept falling over that range while held-out MAE did not, so the additional terms fit the training data without generalising. The grid was stopped there instead of extended.

Dropout hurts both neural models. Every configuration at dropout .2 was worse than its counterpart at dropout .0, and the gap widened with depth, reaching 120.4 against 104.7 at two layers and 32 hidden units. With 5,328 training windows and at most about 50,000 parameters the models are not overfitting, so regularisation was removing capacity that was being used. The CNN was consequently left at a light .1 instead of being swept upward.

Kernel width matters more than depth or width. Moving the CNN kernel from three to five at a fixed 32 filters improved validation MAE from 122.7 to 109.3, whereas doubling the filters at kernel three reached only 112.1 and doubling the depth was worse. This is a receptive-field effect, and it is the same limitation that produces the overnight failure reported below.

Transfer Instead of a Fresh Search

The winning configurations from square 5161 were refit, not re-searched, on squares 5059, 5259, 4159, and 4556, with identical splits and an independently fitted train-only scaler per area. This is a deliberate test of hyperparameter transfer across traffic regimes, and it is also a limitation: a per-area search would likely improve every non-5161 number, so the reported cross-area results are a lower bound.

A Methodological Asymmetry

SARIMA was refit on the combined training and validation data before forecasting the test week, because its hyperparameter selection does not consume the validation set in a way that would leak. The LSTM and CNN were trained on the training split only, because the validation

set is what stops them. The neural models therefore saw 5,328 training windows against SARIMA's 6,480 observations.

This asymmetry favours SARIMA. It is reported here without being corrected, because correcting it would mean either denying SARIMA data it can legitimately use or taking away the neural models' stopping criterion. The consequence for interpretation is one-directional: every margin the LSTM and CNN hold over SARIMA was achieved on less data, so those margins are conservative.

Timing Protocol and Hardware

Timings are wall-clock measurements taken in-process, with training and inference recorded separately. Training time is one complete fit of the selected configuration; for the neural models this is the full early-stopped run including per-epoch validation, not the time to the best epoch. Inference time is a single pass producing all 1,008 one-step test predictions, and the per-slot figure is that divided by 1,008. Each figure is a single run, so run-to-run variance is not characterised; the spread across areas is the best available indication of it.

All training ran on the central processing unit under macOS Darwin 24.6.0 on an Intel Core i7-8569U at 2.80 GHz with four physical and eight logical cores, 16 GiB of memory, and Python 3.9.6. No graphics processor was used, which is why the LSTM figures are as large as they are.

Results and Discussion

The Three Required Areas

Tables 9 through 11 report every model on the three highest-traffic areas over 16 to 22 December 2013.

Table 9

Forecasting Performance on Square 5161, the Highest-Traffic Area

Model	MAE	MAPE (%)	RMSE	Train (s)	Infer (s)
Persistence	92.80	9.19	134.88	0.000	0.001
Seasonal naive	338.59	25.94	619.04	0.000	0.001
SARIMA	108.91	11.52	162.76	0.516	0.074
LSTM	86.35	9.73	126.75	151.449	0.292
1D-CNN	101.72	14.24	140.37	3.884	0.012

Note. Evaluation week 16 to 22 December 2013, 1,008 one-step forecasts. The first two rows are zero-parameter references and are excluded from the marking of the best model. Bold indicates the best of the three selected models.

Table 10

Forecasting Performance on Square 5059, the Second Highest-Traffic Area

Model	MAE	MAPE (%)	RMSE	Train (s)	Infer (s)
Persistence	81.52	7.96	114.38	0.000	0.001
Seasonal naive	171.74	18.02	245.87	0.000	0.001
SARIMA	91.11	9.16	129.27	0.494	0.078
LSTM	78.06	7.53	113.31	45.377	0.393
1D-CNN	79.48	9.95	108.08	2.035	0.010

Note. The CNN takes RMSE on this area, which means its errors are less heavy-tailed than the LSTM's even though its mean error is larger.

Table 11

Forecasting Performance on Square 5259, the Third Highest-Traffic Area

Model	MAE	MAPE (%)	RMSE	Train (s)	Infer (s)
Persistence	75.97	8.11	109.58	0.000	0.001
Seasonal naive	470.32	71.62	861.62	0.000	0.001
SARIMA	88.02	12.06	123.67	0.431	0.071
LSTM	68.06	7.80	96.55	59.047	0.343
1D-CNN	74.80	11.13	99.42	7.668	0.010

Note. The seasonal-naive figure is the worst of any area, which the regime analysis explains: 5259 is the office area, so predicting Monday morning from Sunday morning is wrong by a factor of five.

The LSTM had the lowest MAE and MAPE on all three areas. The order by MAE was LSTM, then 1D-CNN, then SARIMA on every area, which is stable; the order by MAPE was

not, because the CNN's percentage error is inflated by the failure discussed below.

What the References Say

Reading Tables 9 through 11 against persistence changes the conclusion materially.

Table 12

Skill Against the Persistence Reference, by Area

Area	SARIMA	LSTM	1D-CNN	Seasonal naive
5161	−17.4%	+7.0%	−9.6%	−264.9%
5059	−11.8%	+4.2%	+2.5%	−110.7%
5259	−15.9%	+10.4%	+1.5%	−519.1%
4159	−21.1%	+6.1%	+2.6%	−220.9%
4556	−24.3%	+6.9%	−14.7%	−164.5%

Note. Percentage improvement in MAE over the persistence forecast. Positive values mean the model beats predicting no change; negative values mean it does not.

SARIMA lost to persistence on all five areas, by 12% to 24% of MAE. The 1D-CNN lost on two of five. Only the LSTM won everywhere, by 4.2% to 10.4%. On MAPE the picture is harsher: the LSTM is ahead on four areas but behind persistence on 5161, at 9.73% against 9.19%, and the CNN is behind on all five.

This is not a reason to discard SARIMA. The brief asks for three models compared, and SARIMA is the classical reference that the literature treats as necessary [1], [2], [3]. It is a reason to read the comparison correctly. At a ten-minute horizon the immediately preceding observation is an extremely strong predictor, and SARIMA's formulation works against that fact: differencing at lag 144 anchors the forecast on yesterday's value at the same slot and then corrects with an autoregressive moving-average term, instead of anchoring on $x(t)$.

The seasonal-naive row makes the same point in extreme form. A lag-144 autocorrelation of .878 sounds strong, but as a forecaster it is 3.6 times worse than persistence on 5161 and 6.2 times worse on 5259, because a day-old value cannot track the last ten minutes. High autocorrelation at a lag does not make that lag a good one-step predictor when a much closer

observation is available. On 5259 the worst single seasonal-naive error, on 16 December at 01:40, is 856%.

Training and Execution Cost

Table 13

Wall-Clock Cost Averaged Across the Three Required Areas

Model	Train (s), mean	Train range	Infer (s), 1,008 slots	Per slot
SARIMA	0.480	0.431 to 0.516	0.0741	0.074 ms
LSTM	85.291	45.4 to 151.4	0.3426	0.340 ms
1D-CNN	4.529	2.0 to 7.7	0.0106	0.011 ms

Note. Single runs on the hardware described in the methodology. Training time for the neural models includes per-epoch validation.

The LSTM cost roughly 19 times the CNN's training time and 178 times SARIMA's, and its training time varied by a factor of 3.3 across areas purely because early stopping fired at different epochs, 18 on 5161 against 5 on 5059. Inference was cheap for every model: the slowest needs 0.34 ms per forecast, so at a ten-minute cadence any of these runs comfortably in real time on a single core. Inference cost does not separate these models. Training cost does.

Table 14

Accuracy Gain of the LSTM Over the 1D-CNN Against Training-Cost Multiple

Area	LSTM MAE gain over CNN	Train-time ratio
5161	15.1%	39.0×
5259	9.0%	7.7×
4556	18.8%	19.7×
5059	1.8%	22.3×
4159	3.6%	40.2×

Note. The gain is neither proportional to the cost nor stable across areas.

On square 5059 the LSTM bought 1.8% of MAE for 22 times the training time; on 4556 it bought 18.8% for 20 times. If a model must be maintained per cell across 10,000 cells, the

CNN is the defensible default and the LSTM is worth reserving for cells where a measured gain justifies it. If only a handful of hotspots matter, the LSTM's 85 s is irrelevant and it should simply be used.

Failure Analysis

Where the 1D-CNN Breaks: The Night Trough on Square 5161

On square 5161 the CNN beat SARIMA on MAE, at 101.7 against 108.9, but lost badly on MAPE, at 14.24% against 11.52%. The gap is not in the daytime peaks.

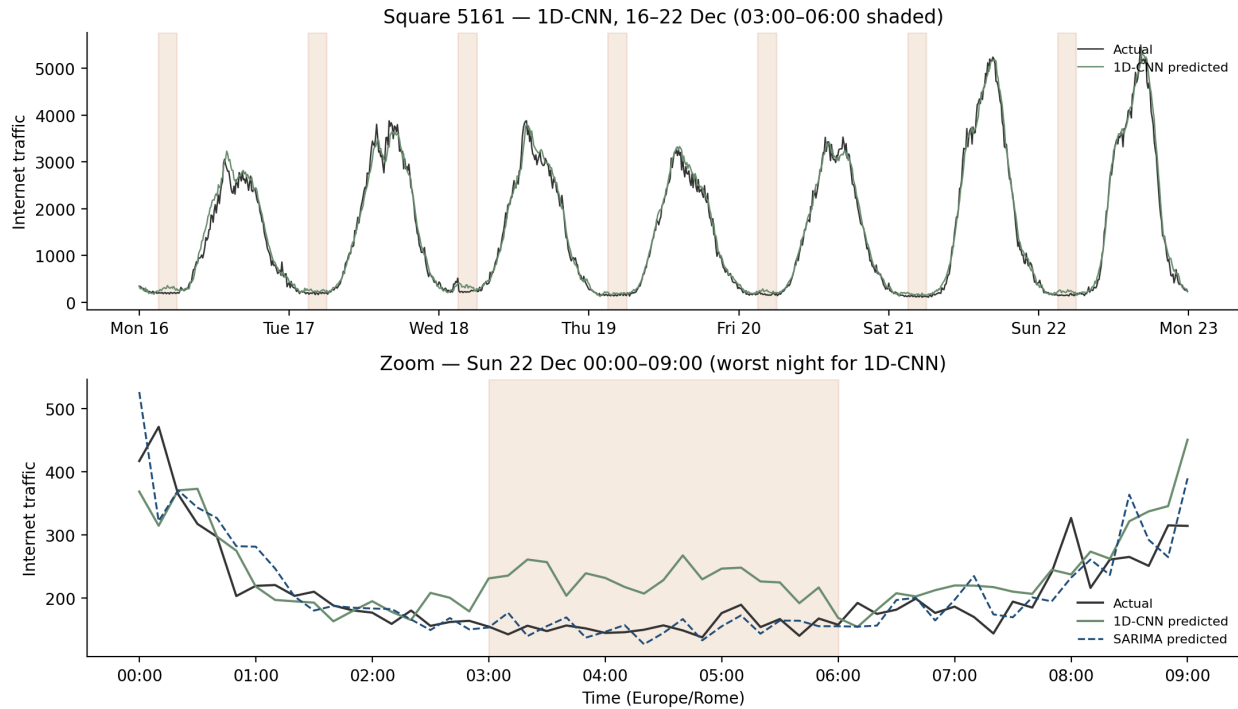
Table 15

Mean Absolute Percentage Error by Time of Day on Square 5161

Model	03:00 to 05:00	09:00 to 17:00	Share of error mass, 03 to 05
Persistence	10.37%	6.22%	14.1%
SARIMA	15.15%	7.58%	16.4%
LSTM	11.38%	5.83%	14.6%
1D-CNN	36.82%	6.79%	32.3%

Note. The final column gives each model's share of its own total percentage error falling in three overnight hours.

In daylight the CNN was better than SARIMA, at 6.79% against 7.58%. Three hours of the night carry a third of its total percentage error. At 04:30 on 16 December the true value is 200, the CNN predicts 357, a 79% error, and SARIMA predicts 186, a 7% error.

Figure 10*Overnight Over-Prediction by the 1D-CNN on Square 5161*

Note. Upper panel: the full test week with 03:00 to 06:00 shaded. Lower panel: a zoom on the worst night with SARIMA shown for comparison. The model carries daytime level into the trough, which a percentage metric punishes hardest where the trough is deepest.

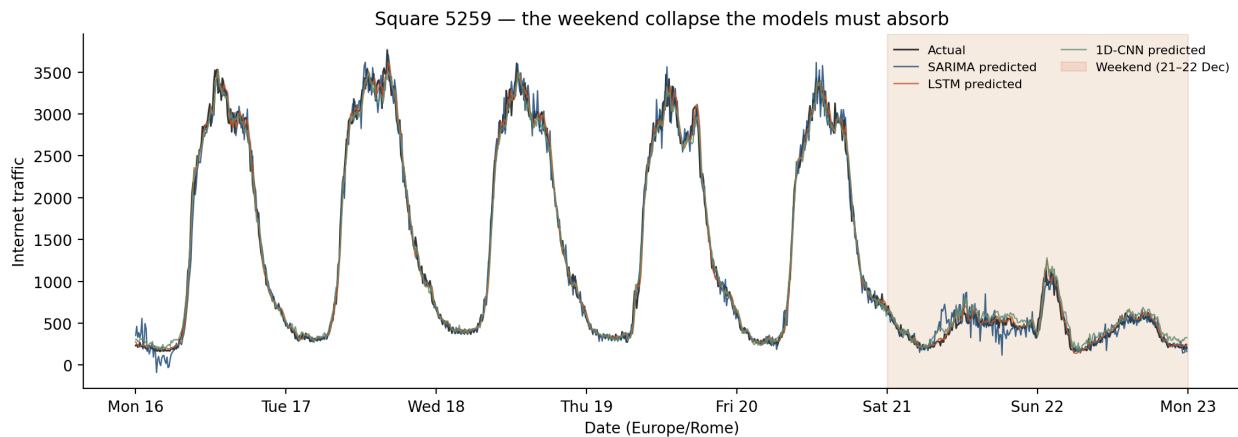
The mechanism is the one predicted by the regime analysis and the model description. The selected CNN has a five-slot, 50-minute receptive field; everything beyond that reaches the output only through fixed weights on a flattened feature map. It therefore tracks the shape of the evening decay poorly and carries residual daytime level into the trough. Square 5161 is where that is most expensive, because it holds only 2.30% of its traffic between 01:00 and 05:00, the deepest trough of any area here, so an absolute over-prediction that would be unremarkable at noon becomes a 79% relative error at 04:30. The prediction is testable and it holds: on square 4556, which keeps 10.90% of its traffic overnight, the CNN's night problem is far milder. The fault is the configuration, not convolution itself: dilated convolutions would reach lag 144 at the same parameter count.

Where Every Model Breaks: Weekends

The worst day of the test week, by mean percentage error, was a Saturday or a Sunday for 13 of the 15 area-model pairs across the three required areas and the two extension areas. On square 5259 the SARIMA and CNN percentage errors roughly quadrupled from their Wednesday best, 5.45% and 4.80%, to their weekend worst, 23.63% and 23.59%.

Figure 11

All Three Models Across the Test Week on Square 5259



Note. The weekend is shaded. The collapse to roughly a fifth of weekday peak is tracked in one-step terms, but it is where every model's percentage error concentrates. SARIMA's jitter, and its negative excursion early on 16 December, are visible.

This is the cost of the sequence-length decision, and the decomposition priced it in advance: the weekly component carries 9.99% of the variance and no model here can observe lag 1,008. A 144-step window at 03:00 on Saturday looks almost exactly like one at 03:00 on Wednesday, and nothing in the input distinguishes them. The models therefore enter each weekend with the wrong prior and correct only as the day's own data arrives. Three architecturally unrelated models fail in the same way on the same days, which is the strongest evidence in this study that the limitation is real.

A Defect Specific to SARIMA

Being an unconstrained linear model applied to a strictly non-negative quantity, SARIMA emitted negative traffic predictions: 5 of 5,040 across all areas, with a minimum of -89.3 on square 5259, visible in Figure 11 early on 16 December. The count is small and the effect on the metrics negligible, but a forecaster that can output physically impossible values needs a clamp before deployment. Neither neural model does this.

Does the Ranking Survive a Different Regime?

Squares 4159 and 4556 ran through the identical pipeline with the hyperparameters selected on 5161.

Table 16

Forecasting Performance on the Two Extension Areas

Area	Metric	Persistence	SARIMA	LSTM	1D-CNN
4159	MAE	15.95	19.32	14.99	15.55
4159	MAPE	6.98%	8.70%	6.51%	7.15%
4556	MAE	28.86	35.86	26.88	33.10
4556	MAPE	6.60%	8.17%	6.10%	8.08%

Note. These areas run at roughly a fifth and a third of the traffic level of the required areas.

The MAE ranking of LSTM, then CNN, then SARIMA held on both, so it survives an order-of-magnitude drop in traffic level and both weekly regimes. Two qualifications apply: the LSTM's margin over the CNN on 4159 shrinks to 3.6%, the smallest of all five areas, and the CNN is beaten by persistence on 4556 by 14.7%. Hyperparameters chosen on one area therefore transfer in rank but not in margin, which is consistent with the site dependence reported by Sone et al. [3].

Design Decisions and Room for Improvement

SARIMA. The design decision to revisit first is the seasonal differencing. Forcing a seasonal difference at period 144 was chosen for tractability, and it is exactly what makes SARIMA lose to persistence, because it discards the anchor at $x(t)$. A SARIMA with a first

difference and no seasonal difference, or an ARIMA fitted to the residual of a seasonal-naive forecast, would likely close most of the gap at similar cost. The model is otherwise the best value in the study: half a second to fit, fully interpretable, and within 20% of the LSTM.

LSTM. The LSTM wins, and this study does not fully explain why it wins. Its advantage is largest on 5161 and 5259, the two areas with the most extreme weekly rhythms, which suggests it is learning within-window context that the CNN's flattened head cannot. The obvious improvement is representational, not architectural: hour-of-day and day-of-week features would supply the weekly information the window cannot carry, at negligible cost. That is worth trying before a larger network.

1D-CNN. The receptive-field limit is a design error, and one that can be pointed at directly; convolution itself is not the problem. Dilated convolutions would reach lag 144 at the same parameter count and would likely remove most of the overnight bias. The model's good daytime MAPE, 6.79% against SARIMA's 7.58%, suggests that the architecture is sound and the configuration is not.

The comparison as a whole. The largest methodological weakness is that hyperparameters were searched on one area while the three required areas are neighbours in one city centre. The study is therefore better evidence about one traffic regime observed three times than about three regimes, and the extension areas are what keep the conclusion from being circular.

Conclusion and Future Work

Three architecturally distinct models were compared for one-step-ahead Internet traffic forecasting on the three highest-traffic areas of the Milan grid over 16 to 22 December 2013, with two lower-traffic areas as a control.

The LSTM is the best-performing model, with the lowest MAE and MAPE on all three required areas and on both extension areas. The quantitative case is Tables 9 through 11; the qualitative case is the regime analysis, which shows that these areas differ sharply in weekly rhythm and night depth, and the transfer result, which shows that the LSTM's ranking survives

that variation while its margin does not. The ordering of LSTM, 1D-CNN, and SARIMA by MAE held on all five areas tested.

Three findings qualify that headline, and they matter more than the ranking. First, SARIMA is beaten by persistence on every area, by 12% to 24% of MAE, and on square 5161 no model beats persistence on MAPE; at a ten-minute horizon the last observation is a very strong predictor, and a model that anchors on the previous day instead of the previous slot starts behind. Second, every model's worst days are weekends, on 13 of 15 area-model pairs, which the decomposition priced in advance at 9.99% of the variance sitting beyond a 144-slot window. Third, the accuracy-to-compute trade-off does not favour the LSTM at scale: 19 times the CNN's training time for gains between 1.8% and 18.8% depending on the area.

Limitations. Hyperparameters were searched on one area only, and the three required areas are adjacent cells of one city centre, not independent regimes. The test week is, by accident of the calendar, an ordinary week: the Christmas collapse begins the day after it ends, so nothing here shows how these models behave through the 87% shutdown visible in Figure 9. Each timing is a single run. SARIMA saw more training data than the neural models.

Future work. Five extensions follow, in the order they would be attempted. First, supply the week: hour-of-day and day-of-week features cost nothing and address the largest measured error source. Second, fix the two identified design defects, dilating the CNN to reach lag 144 and reformulating SARIMA to anchor on $x(t)$ rather than on the lag-144 difference. Third, search hyperparameters per regime instead of per grid position, using the regime classification as a clustering key. Fourth, evaluate through the Christmas shutdown, which is the failure case this evaluation window excludes and the one an operator would most want a forecast for. Fifth, extend to multi-step horizons and spatial neighbours as in Zhang and Patras [4], which a Moran's I of .943 suggests would carry real information, while keeping the univariate comparison intact as the reference.

References

- [1] I. Kochetkova, A. Kushchazli, S. Burtseva, and A. Gorshenin, "Short-term mobile network traffic forecasting using seasonal ARIMA and Holt-Winters models," *Future Internet*, vol. 15, no. 9, Art. no. 290, Sep. 2023, doi: 10.3390/fi15090290.
- [2] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular traffic prediction and classification: A comparative evaluation of LSTM and ARIMA," in *Discovery Science (DS 2019)*, Lecture Notes in Computer Science, vol. 11828. Cham, Switzerland: Springer, 2019, pp. 129-144, doi: 10.1007/978-3-030-33778-0_11.
- [3] S. P. Sone, J. J. Lehtomäki, and Z. Khan, "Wireless traffic usage forecasting using real enterprise network data: Analysis and methods," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 777-797, 2020, doi: 10.1109/OJCOMS.2020.3000059.
- [4] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 18th ACM Int. Symp. Mobile Ad Hoc Netw. Comput. (MobiHoc)*, Los Angeles, CA, USA, Jun. 2018, pp. 231-240, doi: 10.1145/3209582.3209606.
- [5] G. L. Santos, P. Rosati, T. Lynn, J. Kelner, D. Sadok, and P. T. Endo, "Predicting short-term mobile Internet traffic from Internet activity using recurrent neural networks," *Int. J. Netw. Manage.*, vol. 32, no. 3, Art. no. e2191, May 2022, doi: 10.1002/nem.2191.
- [6] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Sci. Data*, vol. 2, Art. no. 150055, 2015, doi: 10.1038/sdata.2015.55.
- [7] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on Loess," *J. Off. Stat.*, vol. 6, no. 1, pp. 3-73, 1990.
- [8] K. Bandara, R. J. Hyndman, and C. Bergmeir, "MSTL: A seasonal-trend decomposition algorithm for time series with multiple seasonal patterns," *Int. J. Oper. Res.*, vol. 4, no. 4, pp. 409-427, 2025, doi: 10.1504/IJOR.2025.145992.

- [9] P. A. P. Moran, "Notes on continuous stochastic phenomena," *Biometrika*, vol. 37, no. 1/2, pp. 17-23, Jun. 1950, doi: 10.2307/2332142.
- [10] Source code and full experiment log: <https://github.com/Keaane/milan-traffic-forecasting>
- [11] Video presentation: *[link to be inserted before submission]*

Appendix A

Required Prediction Overlays

Actual against predicted traffic for each model on each of the three required areas, 16 to 22 December 2013, one-step-ahead.

Figure 12

SARIMA on Square 5161

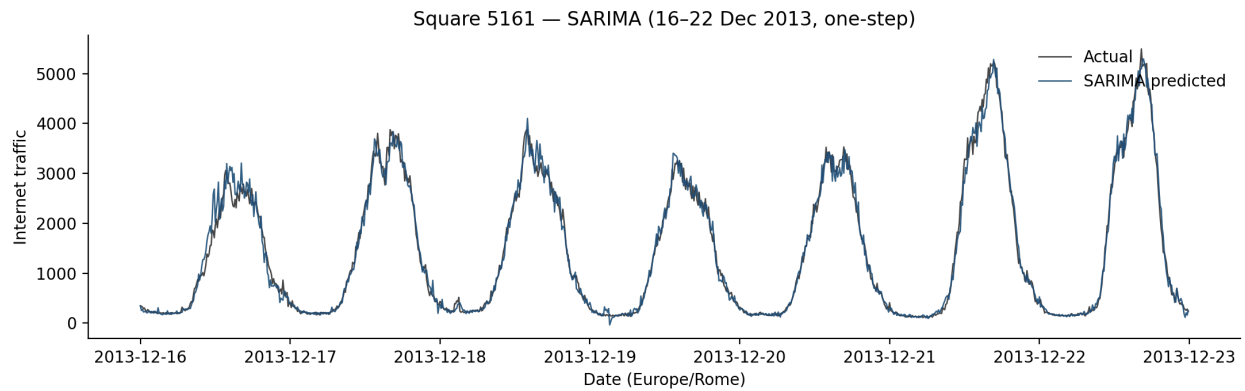


Figure 13

LSTM on Square 5161

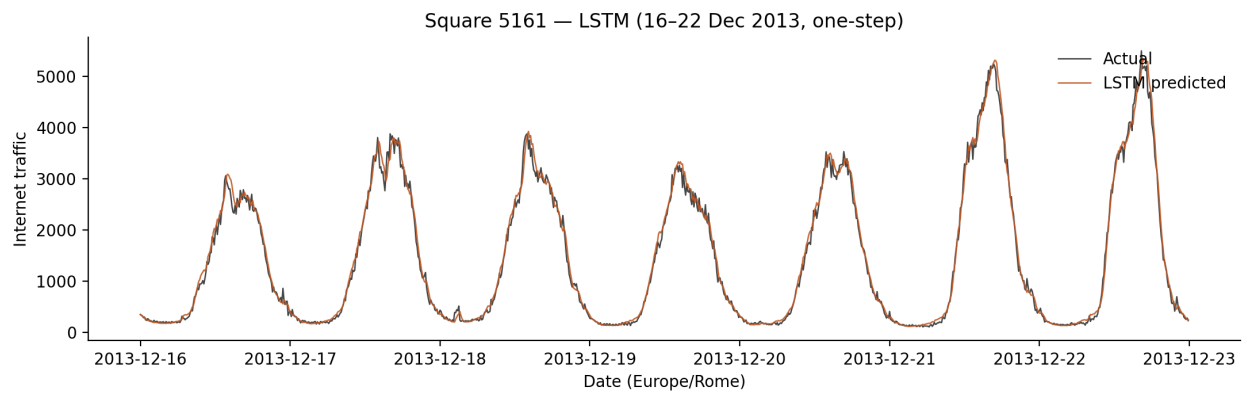


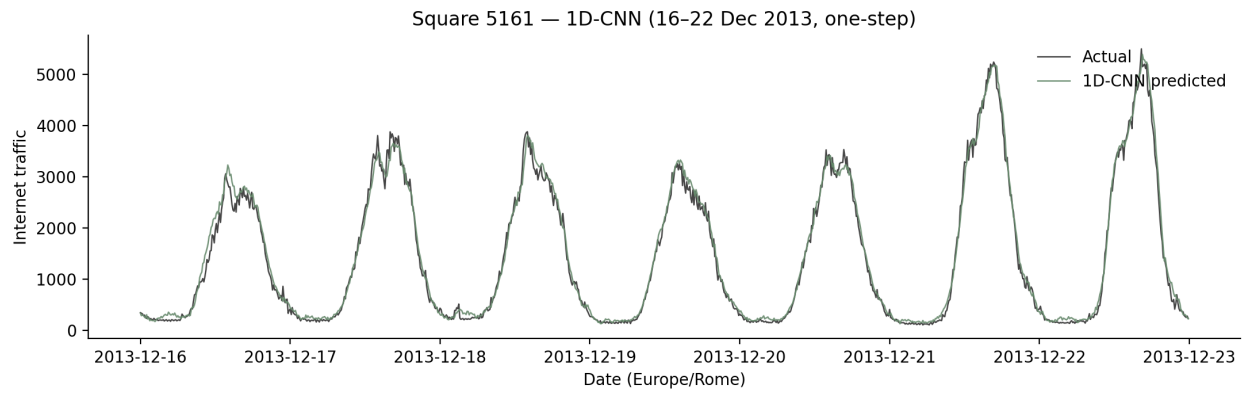
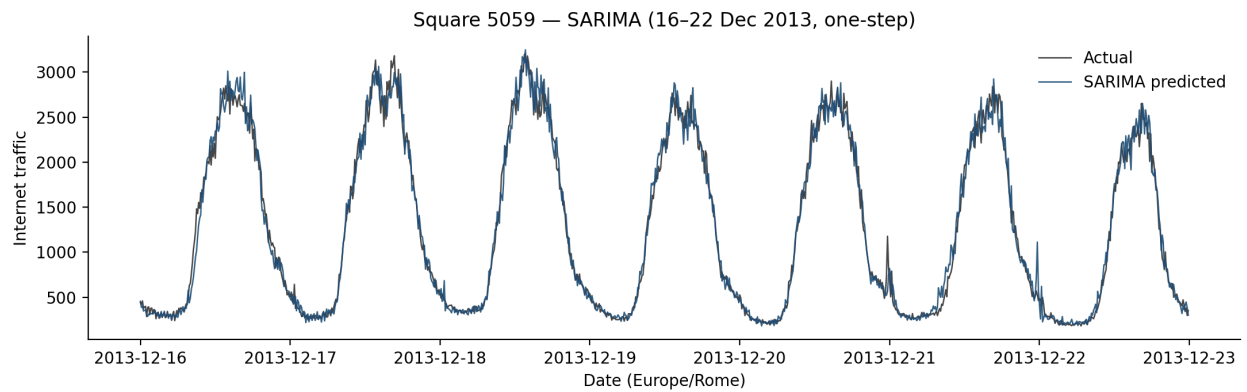
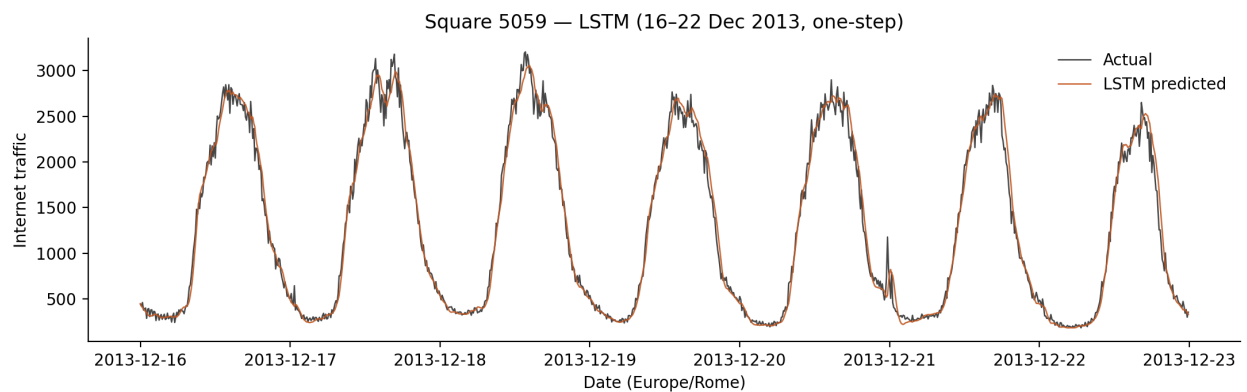
Figure 14*1D-CNN on Square 5161***Figure 15***SARIMA on Square 5059***Figure 16***LSTM on Square 5059*

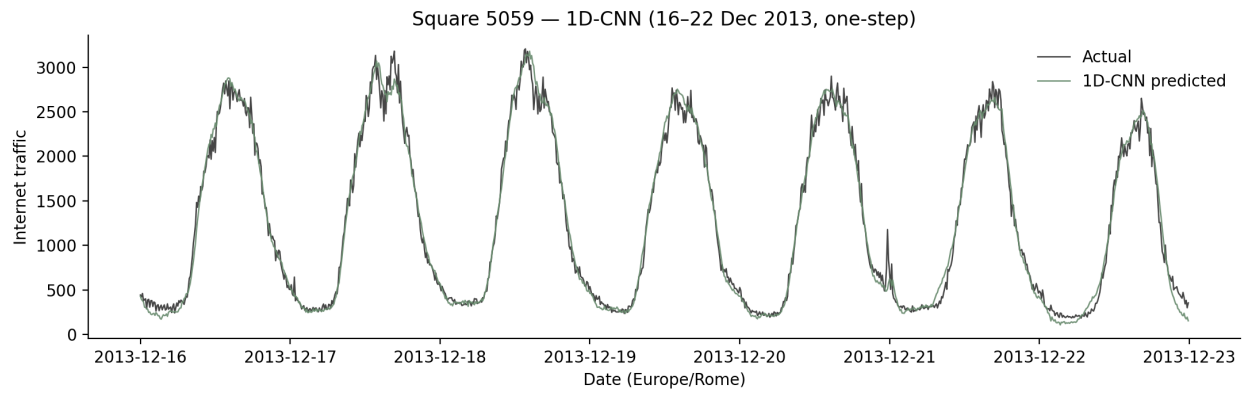
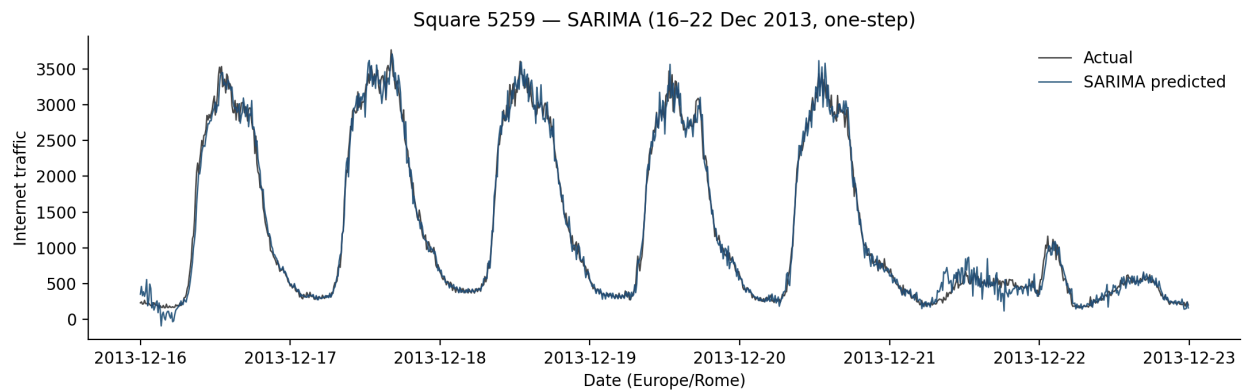
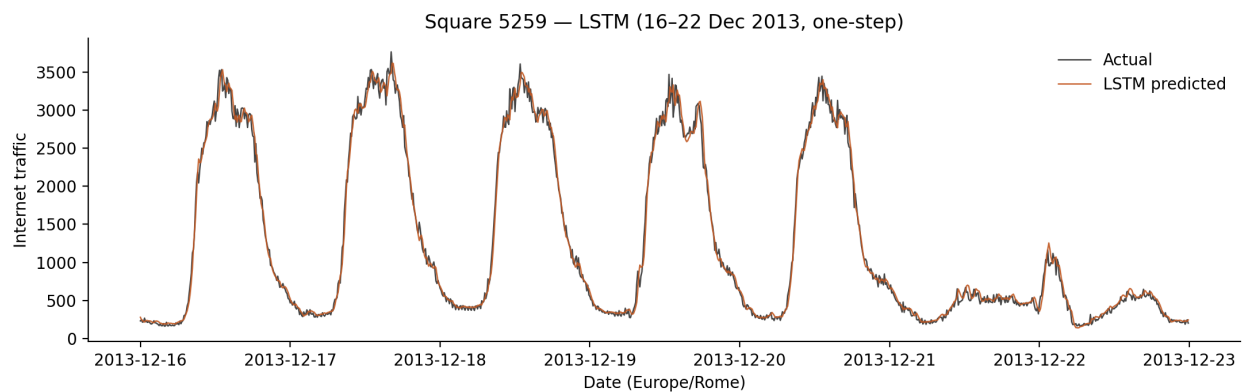
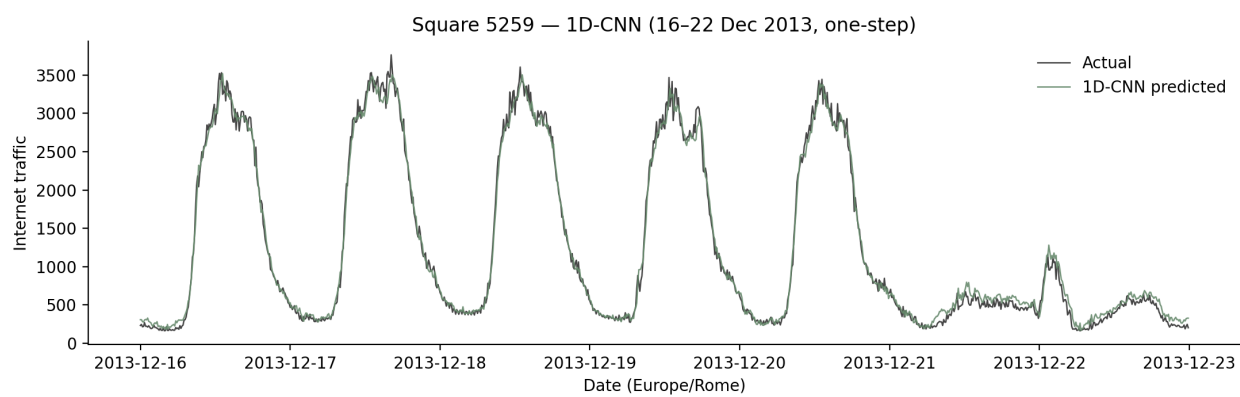
Figure 17*1D-CNN on Square 5059***Figure 18***SARIMA on Square 5259***Figure 19***LSTM on Square 5259*

Figure 20*1D-CNN on Square 5259*

Appendix B

Extension-Area Overlays

Figure 21

SARIMA on Square 4159

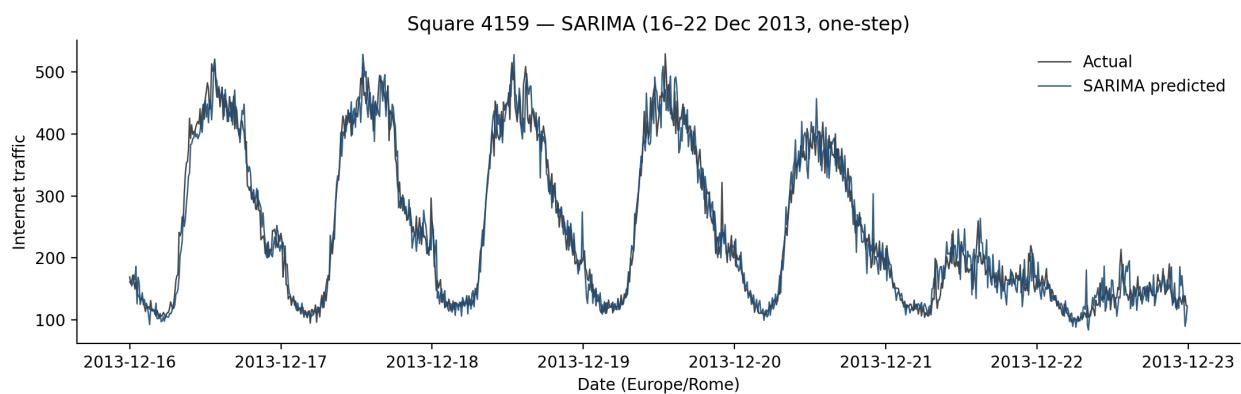


Figure 22

LSTM on Square 4159

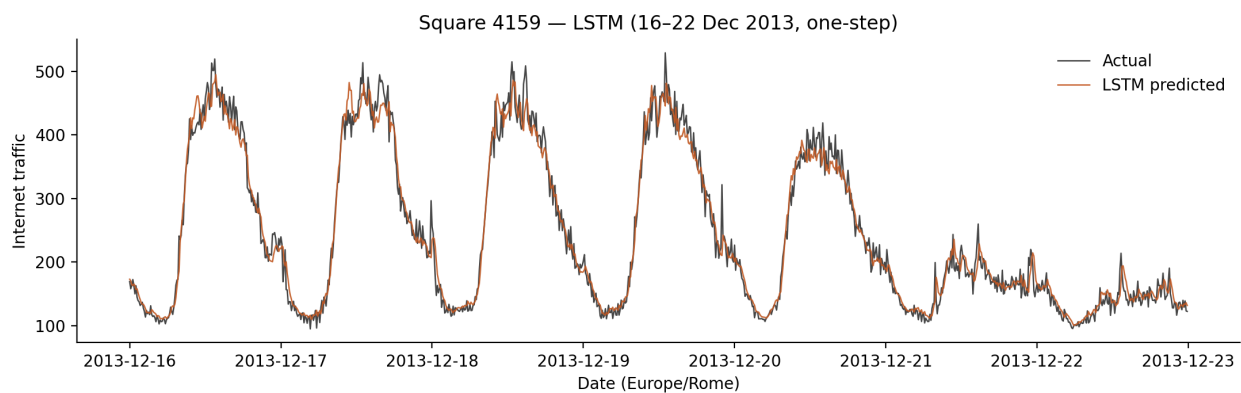


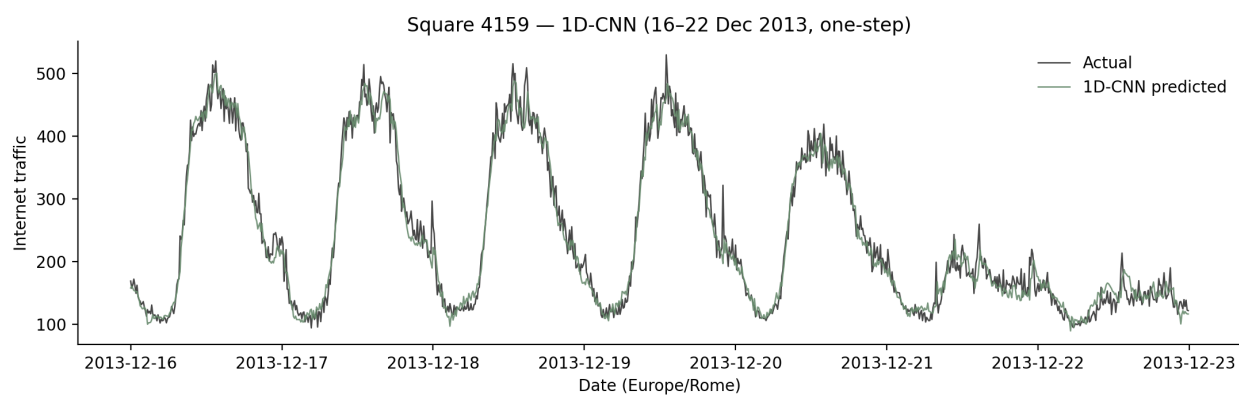
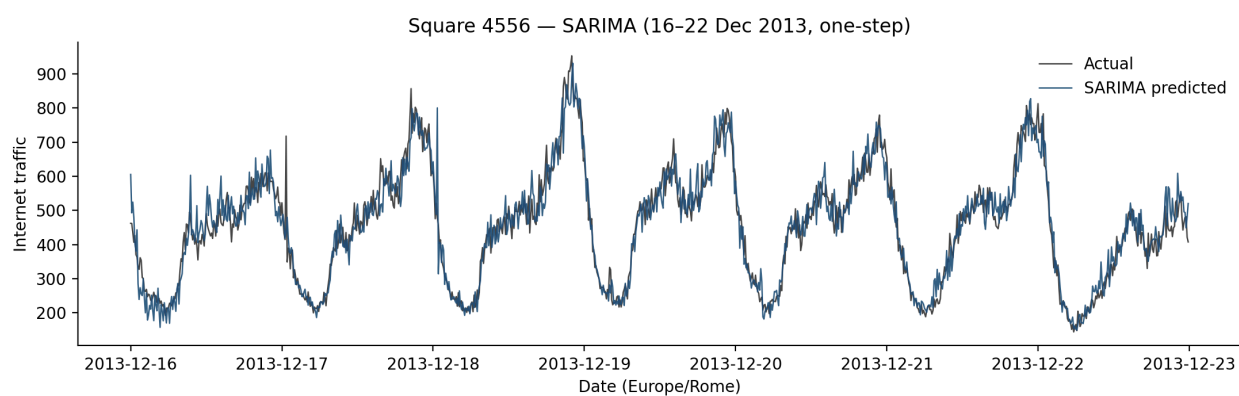
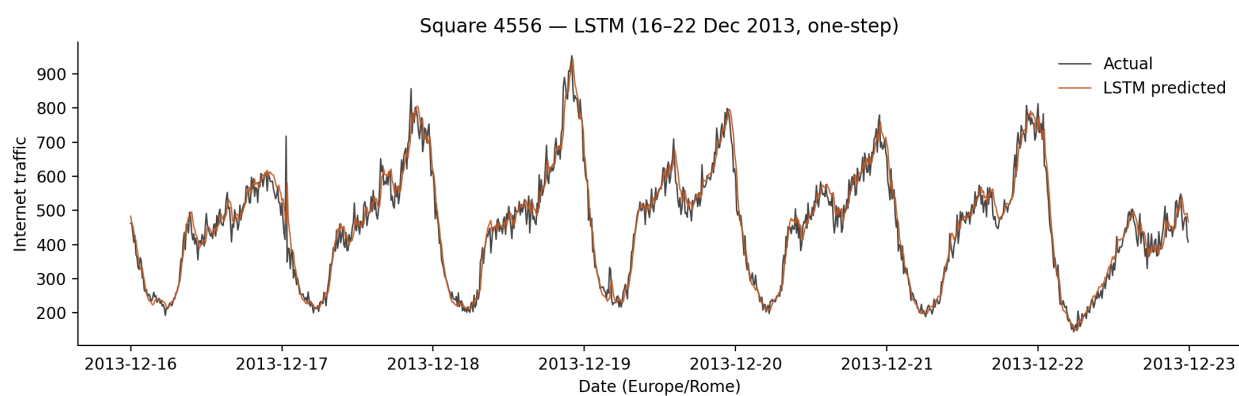
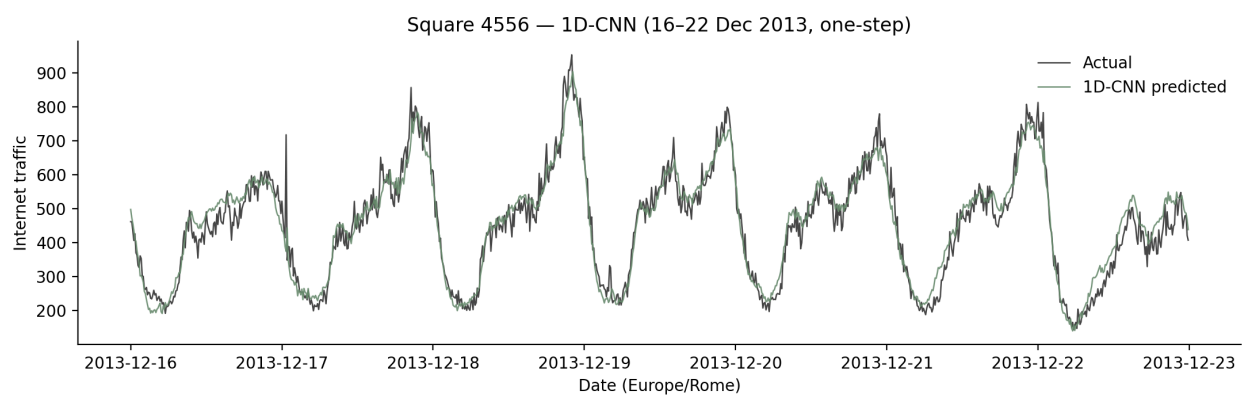
Figure 23*1D-CNN on Square 4159***Figure 24***SARIMA on Square 4556***Figure 25***LSTM on Square 4556*

Figure 26*1D-CNN on Square 4556*

Appendix C

Reproducing These Results

```
python3 -m pip install -r requirements.txt

make bench      # Table 2, and report/memory_benchmark.json
make data       # tidy parquet and per-area totals from data/raw/
make eda        # Figures 1-9, Tables 4-6, report/eda_stats.json
make train      # grid searches on square 5161, Table 8
make evaluate   # refits, references, overlays, Tables 9-11 and 16
make analyze    # Figures 10-11, Tables 12-15, report/results_stats.json
make test       # 74 checks on leakage, metrics and artifact integrity
```

Every number quoted in the exploratory analysis, the results, and the conclusion is written to `report/eda_stats.json`, `report/results_stats.json`, or `report/memory_benchmark.json` by the commands above; none is typed by hand. The test suite re-derives every reported metric from the stored predictions and fails if they disagree.